

Module 223



Réaliser des applications multi-
utilisateurs orientées objets

Réaliser des applications multi-utilisateurs

Objectifs

1.1 Estimer si une base de données remplit les exigences de l'aptitude multi-utilisateurs, et, le cas échéant, documenter les adaptations..

Multi-utilisateurs

Un souci ?

Multi-utilisateurs

Un souci ?

Oui !

En local, un accès concurrent à des ressources peut mener à des incohérences : on l'a vu au 426 (threads).

Solution(s) ? Qui s'en souvient ?

À distance, l'utilisation concurrente de BD (par exemple avec SQL) ou indirectement (via des API qui le feront à notre place), peut poser des problèmes similaires : des incohérences peuvent apparaître si on ne s'en protège pas.

Multi-utilisateurs

Un souci ?

Trois scénarios simples pour bien comprendre le problème :

- réservation de places de cinéma
- virement bancaire
- réservation de places dans des bus de taille limitée

Réservation de places de cinéma



Disponibilités ?

Réponse

Écran								
01	02	03	04	05	06	07	08	09
11	12	13	14	15	16	17	18	19
21	22	23	24	25	26	27	28	29
31	32	33	34	35	36	37	38	39
41	42	43	44	45	46	47	48	49
51	52	53	54	55	56	57	58	59
61	62	63	64	65	66	67	68	69
71	72	73	74	75	76	77	78	79
81	82	83	84	85	86	87	88	89
91	92	93	94	95	96	97	98	99

...
ensuite

...

Réservation places
N°65+66

BD Cinéma

API



Temps

Temps



Disponibilités ?

Réponse

Écran								
01	02	03	04	05	06	07	08	09
11	12	13	14	15	16	17	18	19
21	22	23	24	25	26	27	28	29
31	32	33	34	35	36	37	38	39
41	42	43	44	45	46	47	48	49
51	52	53	54	55	56	57	58	59
61	62	63	64	65	66	67	68	69
71	72	73	74	75	76	77	78	79
81	82	83	84	85	86	87	88	89
91	92	93	94	95	96	97	98	99

...
ensuite

...

Réservation places
N°65+66

BD Cinéma

API



Temps

Temps

Tout va bien, aucun problème dans ce scénario !



Disponibilités ?

Réponse

Écran									
01	02	03	04	05	06	07	08	09	
11	12	13	14	15	16	17	18	19	
21	22	23	24	25	26	27	28	29	
31	32	33	34	35	36	37	38	39	
41	42	43	44	45	46	47	48	49	
51	52	53	54	55	56	57	58	59	
61	62	63	64	65	66	67	68	69	
71	72	73	74	75	76	77	78	79	
81	82	83	84	85	86	87	88	89	
91	92	93	94	95	96	97	98	99	

...
ensuite

...

Réservation places
N°65+66

BD Cinéma

API



Réservation places
N°64+65



Temps

Temps



Disponibilités ?

Réponse

Écran									
01	02	03	04	05	06	07	08	09	
11	12	13	14	15	16	17	18	19	
21	22	23	24	25	26	27	28	29	
31	32	33	34	35	36	37	38	39	
41	42	43	44	45	46	47	48	49	
51	52	53	54	55	56	57	58	59	
61	62	63	64	65	66	67	68	69	
71	72	73	74	75	76	77	78	79	
81	82	83	84	85	86	87	88	89	
91	92	93	94	95	96	97	98	99	

...
ensuite

...

Réservation places
N°65+66

BD Cinéma

API



Réservation places
N°64+65



Oups, la place N°65 n'est plus disponible ! Elle ne peut pas être vendue 2x !!

Temps

Temps

Virement bancaire



Temps

API



BD Banque

t_client		
pk_client	nom	prenom
10	FRIEDLI	Paul
20	HARONI	Mac
30	TERRIEUR	Alain

t_compte			
pk_compte	fk_client	iban	solde
1	10	CH1234123412341234	1'000'000
2	20	CH5678567856785678	20'000
3	30	CH9090909090909090	50'000



Souhait : Paul veut virer 5000.- à Alain

```
{  
  "operation": "virement",  
  "pk_compte_source": 1,  
  "pk_compte_destination": 3,  
  "montant": 5000.0  
}
```

input

output

```
{  
  "resultat": true,  
  "execution_le": "25.12.2025 23:59:59",  
  "montant": 5000.0,  
  "etat_precedant": {  
    "solde_source": "1000000.00",  
    "solde_destination": "50000.00"  
  },  
  "etat_actuel": {  
    "solde_source": "995000.00",  
    "solde_destination": "55000.00"  
  }  
}
```

Temps

BD Banque



t_client		
pk_client	nom	prenom
10	FRIEDLI	Paul
20	HARONI	Mac
30	TERRIEUR	Alain

t_compte			
pk_compte	fk_client	iban	solde
1	10	CH1234123412341234	1'000'000
2	20	CH5678567856785678	20'000
3	30	CH9090909090909090	50'000

Quelles opérations sont nécessaires au niveau BD afin de fournir le résultat escompté ?



Souhait : Paul veut virer 5000.- à Alain

```
{
  "operation": "virement",
  "pk_compte_source": 1,
  "pk_compte_destination": 3,
  "montant": 5000.0
}
```

input

output

```
{
  "resultat": true,
  "execution_le": "25.12.2025 23:59:59",
  "montant": 5000.0,
  "etat_precedant": {
    "solde_source": "1000000.00",
    "solde_destination": "50000.00"
  },
  "etat_actuel": {
    "solde_source": "995000.00",
    "solde_destination": "55000.00"
  }
}
```

```
{
  "resultat": false,
  "erreur": "Montant insuffisant"
}
```

API



BD Banque

t_client		
pk_client	nom	prenom
10	FRIEDLI	Paul
20	HARONI	Mac
30	TERRIEUR	Alain

t_compte			
pk_compte	fk_client	iban	solde
1	10	CH1234123412341234	1'000'000
2	20	CH5678567856785678	20'000
3	30	CH9090909090909090	50'000

Quelles opérations sont nécessaires au niveau BD afin de fournir le résultat escompté ?

- 1) Nouvelle transaction
- 2) Lire solde du compte source => **SoldeA**
- 3) Lire solde du compte destination => **SoldeB**
- 4) Si le solde source est suffisant => **(SoldeA > montant)?**
 - a) Modifier source => **SoldeA = SoldeA - montant**
 - b) Modifier destination => **SoldeB = SoldeB + montant**
 - c) Commit + Produire **JSON réussi**
- 5) Si le solde source est insuffisant
 - a) Rollback + Produire **JSON échec**

Temps



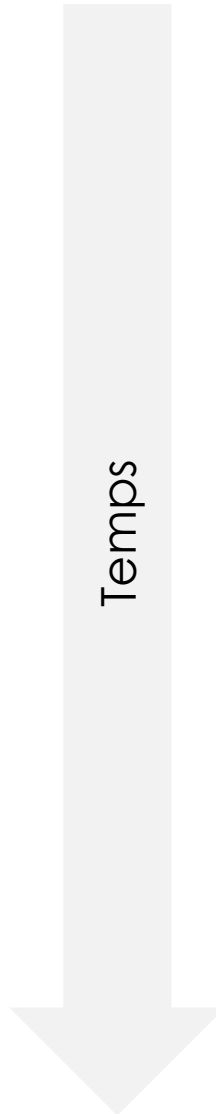
Paul vire 5000.- à Alain



Mac vire 1000.- à Alain



Temps





Paul vire 5000.- à Alain



Mac vire 1000.- à Alain



Et si ces nécessaires opérations se passaient au même moment, que pourrait-il se passer ?

Temps





Paul vire 5000.- à Alain



Mac vire 1000.- à Alain



- 1) Lire solde du compte source
=> SoldeA = 1'000'000
- 2) Lire solde du compte destination
=> SoldeB = 50'000
- 3) Solde suffisant ? => if (SoldeA > 5'000)
 - a) Modifier solde du compte source
=> SoldeA = 995'000
 - b) Modifier solde du compte destination
=> SoldeB = 55'000
 - c) Commit + Produire JSON réussi

- 1) Lire solde du compte source
=> SoldeA = 20'000
- 2) Lire solde du compte destination
=> SoldeB = 50'000

... (le serveur prend ici du temps) ...

- 3) Solde suffisant ? => if (SoldeA > 1'000)
 - a) Modifier solde du compte source
=> SoldeA = 19'000
 - b) Modifier solde du compte destination
=> SoldeB = 51'000
 - c) Commit + Produire JSON réussi

Temps





Paul vire 5000.- à Alain



Mac vire 1000.- à Alain



- 1) Lire solde du compte source
=> SoldeA = 1'000'000
- 2) Lire solde du compte destination
=> SoldeB = 50'000
- 3) Solde suffisant ? => if (SoldeA > 5'000)
 - a) Modifier solde du compte source
=> SoldeA = 995'000
 - b) Modifier solde du compte destination
=> SoldeB = 55'000
 - c) Commit + Produire JSON réussi

- 1) Lire solde du compte source
=> SoldeA = 20'000
- 2) Lire solde du compte destination
=> SoldeB = 50'000

... (le serveur prend ici du temps) ...

- 3) Solde suffisant ? => if (SoldeA > 1'000)
 - a) Modifier solde du compte source
=> SoldeA = 19'000
 - b) Modifier solde du compte destination
=> SoldeB = 51'000
 - c) Commit + Produire JSON réussi

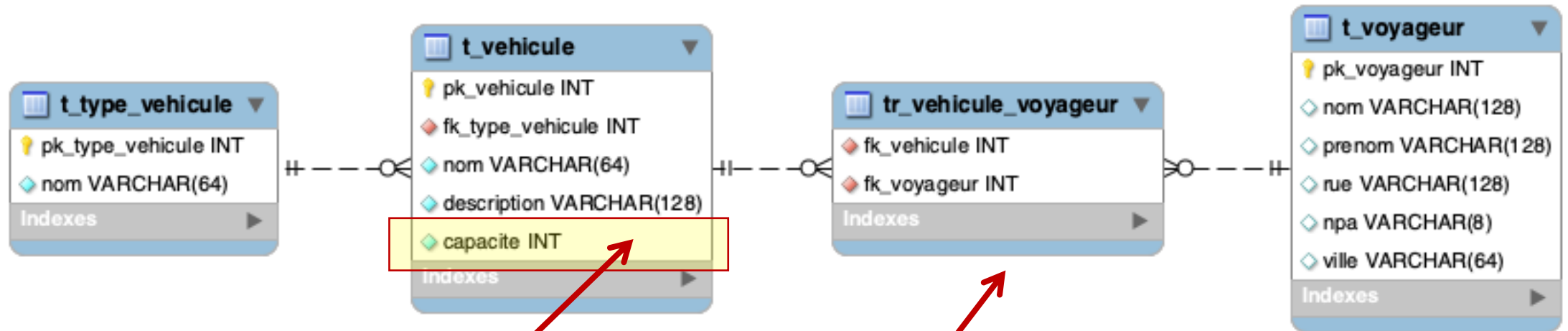
Temps

Oups : il manque 4000 Frs au compte de destination !!!

Réservation de places dans des bus de taille limitée

BD Voyageurs

API



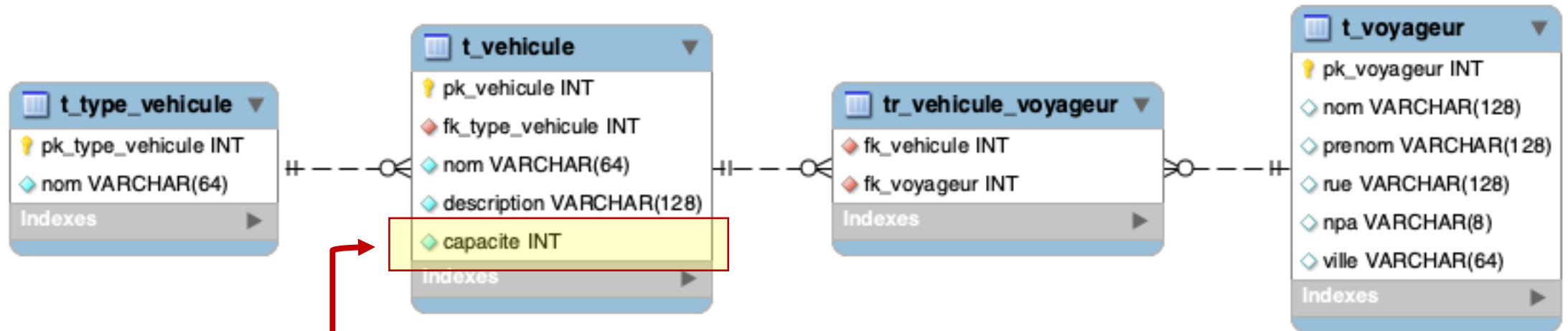
Capacité max d'un bus

Voyageurs dans les bus

Voyageurs

BD Voyageurs

API



Dans un contexte où des dizaines d'utilisateurs peuvent, en même temps, interagir avec la base de données :

Comment garantir, quoiqu'il arrive, le respect de cette capacité ??

Solution ?

Multi-utilisateurs

Que proposent les SGBD ?

Pour gérer ces situations de concurrence d'accès multi-utilisateur et éviter d'éventuelles incohérences, **les SGBD mettent à disposition divers outils** comme notamment les transactions, mais pas seulement !

Il n'y a malheureusement pas de standard et **chaque SGBD a ses propres outils dans ce domaine** :

- par exemple **FOR UPDATE** pour **MySQL**
- **WITH (UPDLOCK)** avec **SQLServer**
- exceptions spécifiques pour MongoDB
- ...

MySQL

Verrouillage collaboratif

SELECT ... FOR SHARE

Verrouille les lignes concernées en permettant à d'autres transactions de lire (**SELECT**), mais pas de mettre à jour (**UPDATE**) ni supprimer (**DELETE**) celles-ci en les bloquant temporairement.

SELECT ... FOR UPDATE

Verrouille les lignes concernées en ne permettant pas à d'autres transactions de lire (**SELECT**)*, de mettre à jour (**UPDATE**) ou supprimer (**DELETE**) celles-ci en les bloquant temporairement.

*ce n'est pas le FOR UPDATE qui empêche la lecture mais le niveau d'isolation. Le for update va empêcher la lecture à condition que l'autre transaction utilise également un for update

Voir : <https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html>

MySQL

Verrouillage collaboratif

, alors voici ce qui arrivera à cette autre requête :					
En clair, si on fait.. ⇓	SELECT ... WHERE pk=1	SELECT ... WHERE pk=1 FOR SHARE	SELECT ... WHERE pk=1 FOR UPDATE	UPDATE ... WHERE pk=1	DELETE ... WHERE pk=1
SELECT ... WHERE pk=1 FOR SHARE	PAS IMPACTEE, S'EXECUTE	PAS IMPACTEE, S'EXECUTE	MISE EN ATTENTE Car veut le verrouillage mais ne peut pas l'obtenir tant que durera le FOR SHARE	MISE EN ATTENTE	MISE EN ATTENTE
SELECT ... WHERE pk=1 FOR UPDATE	PAS IMPACTEE, S'EXECUTE	MISE EN ATTENTE	MISE EN ATTENTE	MISE EN ATTENTE	MISE EN ATTENTE

Multi-utilisateurs

Solution ?

De manière générale, **utiliser les transactions** pour garantir un état complet d'une opération métier : « toutes les actions ont été faites » ou « aucune n'a été faite ».

Lorsqu'une opération métier nécessite plusieurs actions et que leur entrelacement peut mener à des incohérences, on y **verrouille** très temporairement **une ressource critique**, afin d'éviter ces entrelacements problématiques dus aux accès concurrents.

De même, **s'il y a risque d'écraser les modifications d'un autre utilisateur**, utiliser les **techniques de détection de changements concurrents** pour identifier et éviter cela.

Multi-utilisateurs

Solution ?

Question 1

Plusieurs opérations sont-elles nécessaires ?
Leur entrelacement peut-il poser un problème de cohérence ?

oui

Solution

Verrouillage collaboratif sur une ressource clé unique qui représente au mieux cette opération métier

Sérialisation forcée par le SGBD

Question 2

Y a-t-il un risque qu'on puisse écraser les modifications d'un autre utilisateur ?

oui

Solution

Détection du changement avec une **clause WHERE spéciale**, sur un N° de version, sur une date/heure de modification, sur tous les champs du record, ...

Détection de modification par le SGBD

Solution

Verrouillage collaboratif



Paul vire 5000.- à Alain



Mac vire 1000.- à Alain



Le verrouillage collaboratif

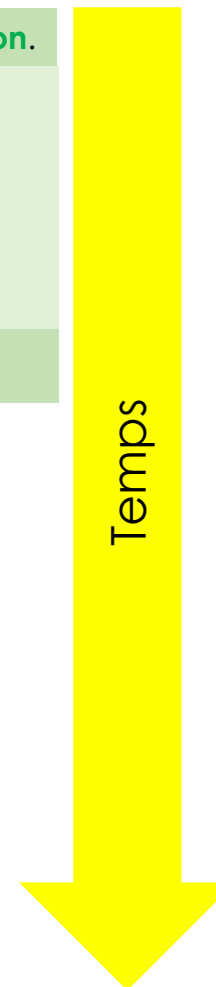
Dans une transaction, demande de **verrouiller source et destination.**

- 1) Lire solde du compte source
- 2) Lire solde du compte destination
- 3) Solde suffisant ?
 - a) Modifier solde du compte source
 - b) Modifier solde du compte destination
 - c) Commit + Produire JSON réussi

Fin de la transaction et déverrouillage

```
SELECT pk_compte
FROM t_compte
WHERE pk_compte IN (1,3)
FOR UPDATE
```

Temps





Paul vire 5000.- à Alain



Mac vire 1000.- à Alain



Le verrouillage collaboratif

Dans une transaction, demande de **verrouiller source et destination.**

- 1) Lire solde du compte source
- 2) Lire solde du compte destination
- 3) Solde suffisant ?
 - a) Modifier solde du compte source
 - b) Modifier solde du compte destination
 - c) Commit + Produire JSON réussi

Fin de la transaction et déverrouillage

```
SELECT pk_compte
FROM t_compte
WHERE pk_compte IN (1,3)
FOR UPDATE
```

Dans une transaction, demande de **verrouiller source et destination.**

Mise en attente par le SGBD car **ces verrous ne sont pas encore disponibles...**

```
SELECT pk_compte
FROM t_compte
WHERE pk_compte IN (2,3)
FOR UPDATE
```

Temps



Paul vire 5000.- à Alain



Mac vire 1000.- à Alain



Le verrouillage collaboratif

Dans une transaction, demande de **verrouiller source et destination**.

- 1) Lire solde du compte source
- 2) Lire solde du compte destination
- 3) Solde suffisant ?
 - a) Modifier solde du compte source
 - b) Modifier solde du compte destination
 - c) Commit + Produire JSON réussi

Fin de la transaction et déverrouillage

Dans une transaction, demande de **verrouiller source et destination**.

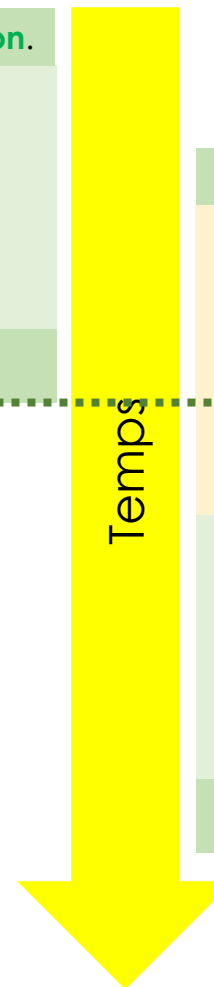
Mise en attente par le SGBD car **ces verrous ne sont pas encore disponibles...**

Source et destination sont maintenant disponibles et sont verrouillés pour cette transaction : le traitement peut se poursuivre...

- 1) Lire solde du compte source
- 2) Lire solde du compte destination
- 3) Solde suffisant ?
 - a) Modifier solde du compte source
 - b) Modifier solde du compte destination
 - c) Commit + Produire JSON réussi

Fin de la transaction et déverrouillage

Temps





Paul vire 5000.- à Alain



Mac vire 1000.- à Alain



Le verrouillage collaboratif

Dans une transaction, demande de **verrouiller source et destination**.

- 1) Lire solde du compte source
- 2) Lire solde du compte destination
- 3) Solde suffisant ?
 - a) Modifier solde du compte source
 - b) Modifier solde du compte destination
 - c) Commit + Produire JSON réussi

Fin de la transaction et déverrouillage

Dans une transaction, demande de **verrouiller source et destination**.

Mise en attente par le SGBD car **ces verrous ne sont pas encore disponibles...**

Source et destination sont maintenant disponibles et sont verrouillés pour cette transaction : le traitement peut se poursuivre...

- 1) Lire solde du compte source
- 2) Lire solde du compte destination
- 3) Solde suffisant ?
 - a) Modifier solde du compte source
 - b) Modifier solde du compte destination
 - c) Commit + Produire JSON réussi

Fin de la transaction et déverrouillage

Temps

Cool ! Code identique mais la portion protégée est automatiquement mise en attente par le SGBD !

Solution

Détection de mise à jour

La détection de mise à jour (ici exemple avec « version »)



Utilisateur A lit des données

t_client				
pk_client	nom	prenom	...	version
1	FRIEDLI	Paul	...	1
2	HARONI	Mac	...	4
3	TERRIEUR	Alain	...	2

Utilisateur A modifie une donnée

```
UPDATE t_client
SET prenom="Max", version=version+1
WHERE (pkclient=2) AND (version=4)
```

Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0



Utilisateur B lit des données

t_client				
pk_client	nom	prenom	...	version
1	FRIEDLI	Paul	...	1
2	HARONI	Mac	...	4
3	TERRIEUR	Alain	...	2

Utilisateur B modifie une donnée

```
UPDATE t_client
SET nom="Imilien", version=version+1
WHERE (pkclient=2) AND (version=4)
```

Query OK, 0 rows affected (0.00 sec)
Rows matched: 0 Changed: 0 Warnings: 0

Temps

Si la clause WHERE n'arrive pas à mettre la main dessus => il a été modifié/supprimé depuis la dernière lecture (=> refresh nécessaire)

Démonstrations

Démonstrations

Voir le problème en action pour bien le comprendre, puis sa solution

Projet VSC **223-demo-verrouillage** qui montre :

- le verrouillage d'un simple record t_personne
- le verrouillage de 2 comptes IBAN (source et destination) pour garantir un traitement complet et correct de ceux-ci
- la réservation de places limitées dans un véhicule

Compléments théoriques

Compléments théoriques

- MySQL : shared locks, update locks,...

<https://dev.mysql.com/doc/refman/8.4/en/innodb-locking-reads.html>

- SQLServer : shared locks, exclusive locks, update locks =>

<https://towardsdev.com/how-to-manage-transactions-and-locking-in-sql-11-44e9eea2ab29>

- MongoDB : (problème) <https://www.mongodb.com/blog/post/how-to-select-for-update-inside-mongodb-transactions>, (et solution possible)

<https://www.mongodb.com/docs/manual/core/transactions-in-applications/#std-label-unknown-transaction-commit-result>